Why Do We Use Activity Diagram

1. It shows the system behavior step by step. Rather than describing the process in paragraphs, the diagram makes the sequence of actions immediately visible. Anyone reading it can follow the flow from top to bottom and understand exactly what happens at each stage.
2. It shows who is responsible for each action. This is why swimlanes are used. Each vertical lane belongs to one actor: Customer, System, Restaurant, or Driver. When an action block appears in a lane, it means that actor is responsible for performing it. This makes responsibilities crystal clear with no ambiguity.
3. It models decision points and alternative flows. The diamond shape in the diagram (Payment valid?) represents a decision. The diagram shows what happens in both the successful path (Yes) and the failure path (No). This directly corresponds to the alternative flows you wrote in your functional requirements.
4. It connects your functional requirements to visual behavior. Every action block in this diagram maps directly to one of FRs. This is what makes the diagram academically valuable — it is not just a picture, it is a visual proof that your requirements are complete and logically ordered.

Customer Lane :
Actions: Open app → Register/Log in → Browse restaurants → Apply filters → Select restaurant → View menu → Add items to cart → Review cart → Proceed to checkout → Enter payment details → (if payment fails) Display error and retry → Rate restaurant → Write review
What this represents: Everything the human user does manually. These are the interactions that trigger the system to respond. This lane directly reflects FR-01 (Login), FR-02 (Browse), FR-03 (View Menu), FR-04 (Cart), FR-05 (Place Order), and FR-10 (Reviews).

System Lane :
Actions: Validate credentials → Generate session token → Fetch restaurants by GPS → Return filtered list → Check if payment is valid → Process payment → Generate order ID → Send confirmation → Notify restaurant → Query available drivers → Assign nearest driver → Send push notification → Update order status to Delivered → Send delivery confirmation → Prompt for review → Save review → Update restaurant rating
What this represents: Everything the software does automatically in the background without the user seeing it directly. This is the largest lane because the system orchestrates all other actors. This lane reflects FR-01, FR-02, FR-05, FR-06, FR-07, FR-09, FR-10, and FR-11.

Restaurant Lane :
Actions: Receive order notification → Confirm order → Prepare food → Mark order as Ready for Pickup
What this represents: The actions performed by the restaurant staff through their dashboard. This lane reflects FR-08 (Restaurant Dashboard & Menu Management). It is a short lane because the restaurant's role in the order flow is focused — they receive, confirm, prepare, and signal readiness.

Driver Lane :
Actions: Receive assignment notification → Accept order → Travel to restaurant → Pick up order → Mark as Out for Delivery → Deliver to customer → Mark as Delivered
What this represents: The physical and app-based actions performed by the delivery driver. This lane reflects FR-09 (Driver Assignment) and FR-06 (Real-Time Order Tracking) because each status update the driver makes feeds directly into the order tracking the customer sees.

Key Elements Explained :

- **start and stop**
  - These are the initial and final nodes of the diagram. start is represented by a filled black circle and marks the exact point where the process begins. stop is represented by a bullseye symbol and marks where the process ends. Every activity diagram must have exactly one start and at least one stop.
- **Action Blocks :action;**
  - Every line written between colons like :Place Order; represents a single action or activity. It is drawn as a rounded rectangle in the final diagram. Each one represents one concrete step performed by the actor in whose swimlane it appears.
- **Decision Diamond if (Payment valid?)**
  - This is the only branching point in your diagram. It represents a condition that can go two ways. The then (Yes) path continues the happy flow toward order confirmation. The else (No) path sends control back to the Customer lane where the error is shown and the payment is retried. After retrying, control returns to the System lane via endif and the flow continues. This models FR-07's alternative flow exactly.
- **Swimlane Transitions**
  - Every time the diagram switches from one lane to another, it means one actor has finished their part and handed off control to another. For example:
  - Customer enters payment details → hands off to System to validate
  - System assigns driver → hands off to Driver to act
  - Driver marks as Delivered → hands off to System to update records
  - System prompts for review → hands off to Customer to rate

These handoffs represent the integration points between your actors and are the most important parts of the diagram for showing how the system coordinates multiple parties simultaneously.

How This Diagram Connects to other Diagrams :

1. **Use Case Diagram :**

Every swimlane action traces back to a use case. "Place Order", "Track Order", "Rate Restaurant" all appear in both.

2. **Context Diagram :**

The data flows in the context diagram (order data, payment data, driver location) are the information being passed at each swimlane handoff in this diagram.

3. **State Machine Diagram :**

The status updates in the Driver and System lanes (Confirmed → Preparing → Out for Delivery → Delivered) are exactly the states modeled in the state machine diagram.

4. **Gantt / PERT :**

The swimlane sequence shows which features depend on each other, informing which development tasks must come before others in your project timeline.